Mikhail Razakov

SOFTWARE ENGINEER · COMPILERS, RUNTIME & PROGRAM ANALYSIS

Moscow / remote · razakov.mikhail@outlook.com · Telegram · GitHub

PROFILE

I develop compiler infrastructure, program analysis tools and runtime components in C#/.NET, C++/LLVM and Rust. At ISP RAS, I work on symbolic computation in SharpChecker; at MCST, I completed a compiler engineering internship. My projects cover LLVM IR transformations, Roslyn CFG/data-flow analysis, CIL and x86-64 code generation, with regression and differential correctness tests.

TECHNICAL SKILLS

- Compilers and runtime: C#/.NET, CIL/CLR, Roslyn, DynamicMethod, Reflection.Emit; C++23, LLVM 22, LLVM IR; Rust, x86-64 SysV ABI
- Analysis and verification: CFG/SSA, data-flow and effect analysis, transformation legality checks, register allocation, phi lowering, differential/regression testing; Python, Git, Linux

EXPERIENCE

ISP RAS · SharpChecker Oct 2026 — present

Working on symbolic computation for C#/.NET static program analysis.

MCST · Compiler Engineering Internship Jul — Aug 2026

C++23/LLVM 22: experimental LICM, conservative transformation legality checks and Python-based before/after behavior comparison.

Client software development · C#/.NET and Unity individual projects since 2023

Built the KeyAuth ASP.NET Core licensing API using EF Core/PostgreSQL, with Unity client integration and subsequent compatibility support.

PROJECTS

UniversalToolchain · C#/.NET compiler/runtime infrastructure

Developing modular language infrastructure with Bytecode/AIR and interpreter/CIL execution paths. A typed LanguagePlan validates module composition and dependencies before runtime; tests compare results between backends.

LLVM Global IV · interprocedural optimization

An experimental LLVM 22 pass promotes selected integer-global accesses inside loops. Effect analysis and PromotionLegality precede IR transformation. Passed 29/29 regression cases on LLVM 22.1.8, including supported and rejected transformations.

DerefAfterNullAnalyzer · Roslyn CFG / data-flow

Built fixed-point null-state propagation across branches, loops and state joins. Passed 18/18 tests on a pinned Git revision. This is a separate project, not a claimed SharpChecker integration.

x86-64 Codegen & Register Allocation Lab · Rust

An experimental backend for a restricted IR: SSA/CFG, liveness, register allocation, spills and phi lowering. Register assignments are checked independently; native results are compared against a reference IR interpreter.

EDUCATION AND RECOGNITION

HSE University, Faculty of Computer Science — Software Engineering, since 2026.

MEPhI Junior competition — first-degree diploma (2025 and 2026). Presented UniversalToolchain at LangDev 2026 — Official recording